

DESENVOLVIMENTO DE SISTEMAS WEB 1

Jair C Leite



Cookies, Sessions, Storage

O que são?

- São mecanismos usados para armazenar dados em aplicações web
- **Cookies** são pequenos arquivos de dados armazenados no navegador do usuário.
 - Utilizado para informações específicas do usuário, como preferências, carrinhos de compras etc.
 - Possui prazo de expiração
- **Sessions** são formas de armazenar dados do usuário no servidor e no cliente (navegador)
 - Server-side session
 - sessionStorage
 - Informação é excluída quando a aba ou browser é fechado.
- **Local Storage** é uma forma de armazenar dados no navegador sem prazo de exclusão

Isso é invasão de privacidade?

- Depende de como é utilizado...
- Pode proporcionar uma melhor experiência do usuário (UX)
- Pode permitir que o servidor monitore as atividades do usuário durante o uso dos serviços de uma empresa e use os dados de forma indevida, anti-ética e ilegal
- Legislação
 - LGPD, GDPR, CCPA

Direitos dos usuários sobre seus dados

- Ter a confirmação de que os dados estão sendo utilizados;
- Ter acesso aos dados;
- Poder corrigir e atualizar dados fornecidos;
- Solicitar anonimização dos dados, impedindo que sejam identificados como indivíduo;
- Solicitar que os dados sejam transferidos a outra organização;
- Pedir a eliminação completa dos dados (de forma irreversível);
- Ser informados sobre compartilhamento de dados entre organizações, caso seja necessário de acordo com o que está previsto em lei;
- Ser informados sobre a possibilidade de não-consentimentos e quais as consequências de não consentir um dado;
- Revogar o consentimento por completo, de forma livre e gratuita;
- Solicitar as motivações por trás de uma decisão. Por exemplo, caso uma empresa de empréstimos valide o crédito de um cliente a partir de um banco de dados, o cliente pode perguntar quais foram os critérios e quais dados usados para essa decisão.

Diferenças entre cookies, local storage e session storage

- Capacidade de armazenamento
 - Cookie 4KB
 - Local Storage 5MB,
 - Session Storage entre 5MB a 10MB
- Local de armazenamento e prazo
 - Cookie – cliente com prazo definido e no servidor para gestão
 - SessionStorage – navegador, válido durante a sessão (aba)
 - LocalStorage – navegador, sem prazo

Exemplos - cookies

- Usando o DOM para criar
 - `document.cookie = "cookieName=cookieValue; expires=expiryDate; path=pathValue";`
- Usando o DOM para ler

```
var cookies = document.cookie;
var cookieArray = cookies.split("; ");
for (var i = 0; i < cookieArray.length; i++) {
    var cookie = cookieArray[i].split("=");
    var cookieName = cookie[0];
    var cookieValue = cookie[1];
    // Do something with the cookie value
}
```

Exemplo - localStorage

- Usando o DOM para criar
 - `localStorage.setItem('preferencia', 'valorDaPreferencia');`
- Usando o DOM para recuperar
 - `var preferencia = localStorage.getItem('preferencia');`
- Para atualizar
 - `localStorage.setItem('preferencia', 'novoValorDaPreferencia');`
- Para remover
 - `localStorage.removeItem('preferencia');`

Exemplos - sessionStorage

- Usando o DOM para criar
 - // When a user logs in or performs any action that requires a session
 - // Create a session identifier and store data in the session
 - `sessionStorage.setItem('sessionId', 'uniqueSessionId');`
 - `sessionStorage.setItem('username', 'JohnDoe');`
- Usando o DOM para ler
 - // Retrieve session data using the session identifier
 - `var sessionId = sessionStorage.getItem('sessionId');`
 - `var username = sessionStorage.getItem('username');`
 - `console.log('Session ID:', sessionId);`
 - `console.log('Username:', username);`

Server-Side Session Storage

```
const express = require('express');  
const session = require('express-session');  
const app = express();
```

```
app.use(session({  
  secret: 'YourSecretKey',  
  resave: false,  
  saveUninitialized: true  
}));
```

```
app.get('/login', (req, res) => {  
  // Assuming successful authentication  
  req.session.username = 'JohnDoe';  
  res.send('Logged in successfully!');  
});
```

```
app.get('/profile', (req, res) => {  
  if (req.session.username) {  
    res.send('Welcome, ' + req.session.username + '!');  
  } else {  
    res.send('You need to log in first.');
```

```
app.get('/update', (req, res) => {  
  req.session.username = 'JaneSmith';  
  res.send('Username updated successfully!');  
});
```

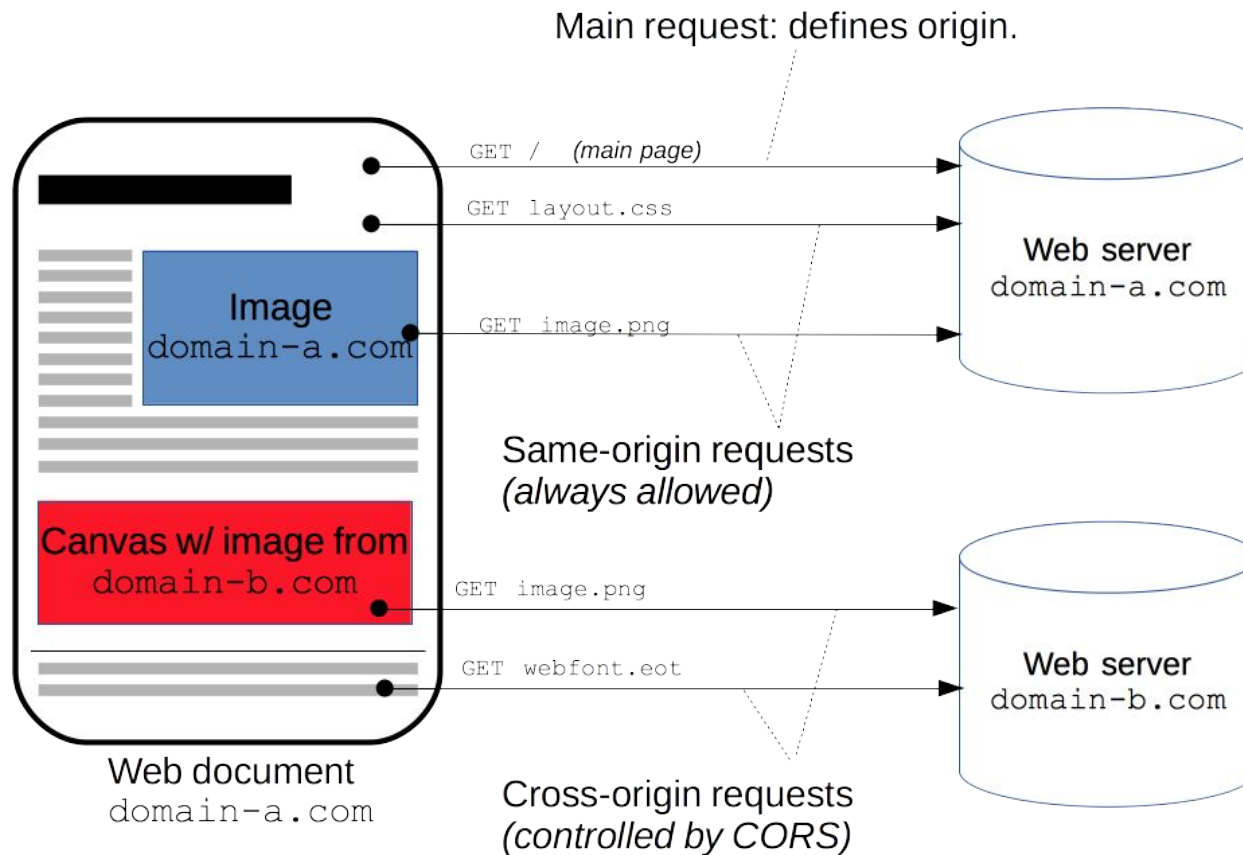
```
app.get('/logout', (req, res) => {  
  req.session.destroy();  
  res.send('Logged out successfully!');  
});
```

Cors - como resolver

CORS - Cross-origin Resource Sharing

- CORS (Cross-origin Resource Sharing) é um mecanismo utilizado pelos navegadores para compartilhar recursos entre diferentes origens
- é uma [especificação do W3C](#) e faz uso de [headers do HTTP](#) para informar aos navegadores se determinado recurso pode ser ou não acessado
- O objetivo é oferecer mais controle, através do protocolo HTTP indicando quais aplicações podem continuar acessando o mesmo servidor.
- Ocorre normalmente com AJAX ou Fetch

CORS - visualização



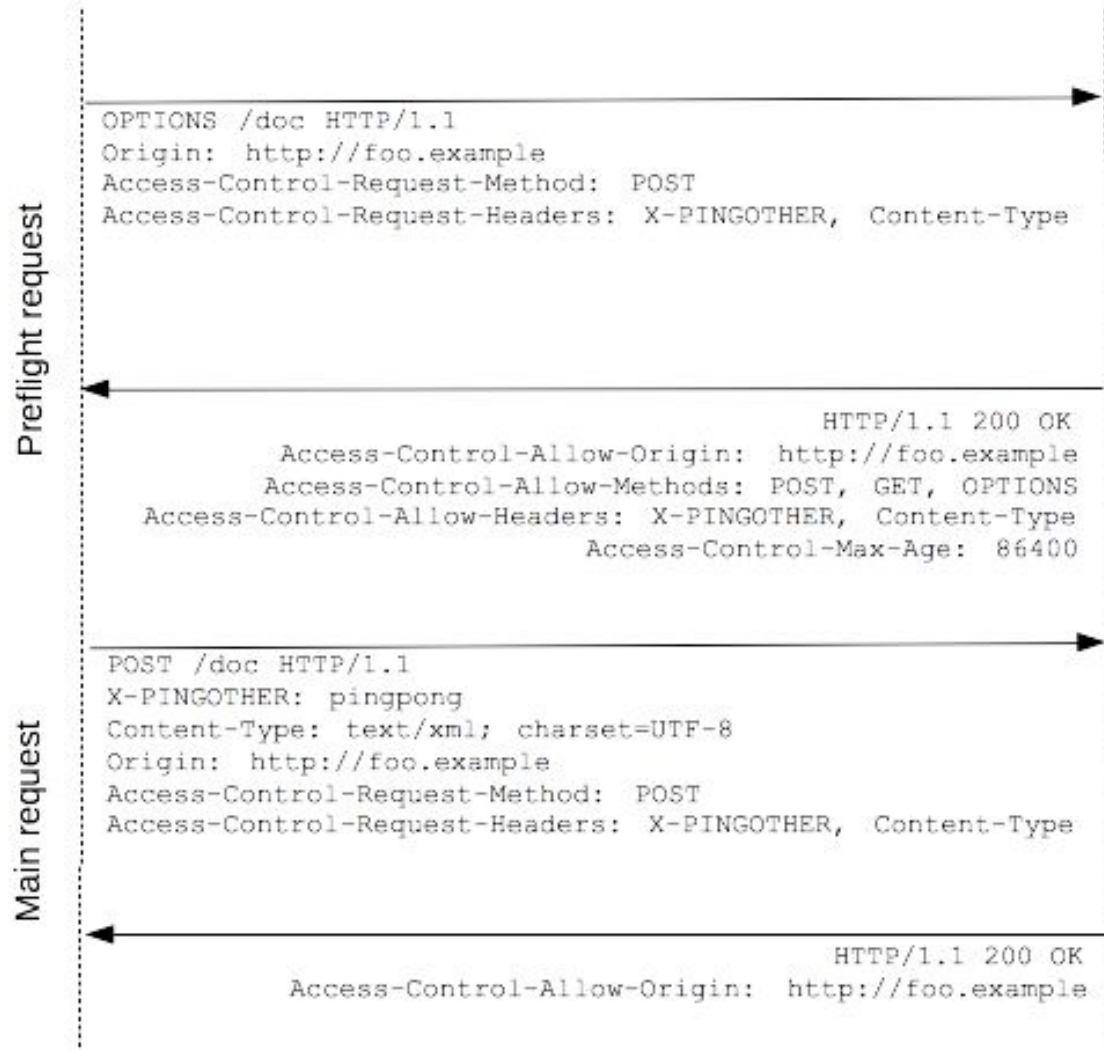
CORS – origens (domínios)

- As seguintes requisições são de mesma origem:
 - <https://painel.treinaweb.com.br/perfil>
 - <http://painel.treinaweb.com.br/>
 - <http://painel.treinaweb.com.br/cursos/>
- São de origens diferentes
 - <http://painel.treinaweb.com.br:81/perfil/index.html>
 - <http://www.treinaweb.com.br/>

Ciclo de requisição com CORS

Client

Server



Como resolver

- Geralmente o problema é causado pela ausência de headers no lado do servidor.
- Para corrigir esse problema, garanta que o servidor está enviando os seguintes headers na sua resposta:
 - Access-Control-Allow-Origin: <https://perfil.treinaweb.com.br>
 - Access-Control-Allow-Methods: POST, GET
 - Access-Control-Allow-Headers: *
 - Access-Control-Max-Age: 86400

Como fazer isso no Express

```
var express = require('express')  
var cors = require('cors')  
var app = express()
```

```
app.use(cors())
```

```
app.get('/products/:id', function (req, res, next) {  
  res.json({msg: 'This is CORS-enabled for all origins!'})  
})
```

```
app.listen(80, function () {  
  console.log('CORS-enabled web server listening on port 80')  
})
```

Configurando o APACHE

- Solicitar ao admin alteração no Apache
 - Header set Access-Control-Allow-Origin "*"

Esse resumo sobre CORS foi baseado em artigo de blog de Gabriel Machado
<https://www.treinaweb.com.br/blog/o-que-e-cors-e-como-resolver-os-principais-erros>